Host a Web Server on the MKR WiFi 1010

Learn how to access your board through a browser on the same network.

Author · Karl Söderby Last revision · 07/17/2024

Introduction

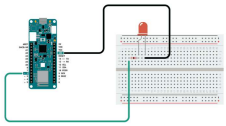
In this tutorial, we will use the MKR WiFi 1010 to set up a simple web server, using the **WiFiNINA** library. The web server will be used as an interface for our board, where we will create two buttons to remotely turn ON or OFF an LED.

Hardware & Software Needed

- Arduino IDE (online or offline).
- WiFiNINA library.
- Arduino MKR WiFi 1010 (link to store).
- Generic LED
- 220 ohm resistor
- Breadboard
- Jumper wires

Circuit

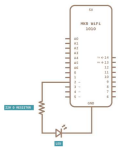
Follow the wiring diagram below to connect the LED to the MKR WiFi 1010 board.



Simple circuit with board, LED and resistor.

Schematic

This is the schematic of our circuit.



Schematics Showing how to connect LED and resistor to the board.

Let's Start

This tutorial barely uses any external hardware, except an LED that we will control remotely. However, the most interesting aspect lies in the library we are going to use: **WiFiNINA**. This library can be used for many different connectivity projects, where it allows us to connect to WiFi, make GET requests and - as

In order to create a web server on our MKR WiFi 1010, we will go through the following steps:

- First, we need to initialize the **WiFiNINA** library.
- Then, we will connect to our local Wi-Fi by entering our `SSID` (name of network) and `PASS` (password of network).
- Once connected, it will start hosting a server, and start waiting for a client.
- If we now enter the IP address of our board in our regular browser (e.g. Chrome, Firefox) we connect as a client.
- As long as we are connected, the program detects it and enters a `while()` loop.
- In the `while()` loop, two links are simply printed in HTML format, which are visible in the browser.
- These links control the LED we connected to the board, by turning it "ON" or "OFF".
- If we press the "ON" link, the program is configured to add an "H" to the end of the URL, or if we press the "OFF" link, we add an "L" to the end of the URL.
- If the URL ends with "H", it will turn on the LED, and if it ends with "L" it turns it off. The "H" stands for "HIGH" and the "L" for "LOW".

And that is the summary of the configuration we will be using in this tutorial. There are a few other functionalities, such as: checking if we have the latest firmware and if we are using the right board, where any of the aforementioned

Creating the Program

1. First, let's make sure we have the drivers installed. If we are using the Cloud Editor, we do not need to install anything. If we are using an offline editor, we need to install it manually. This can be done by navigating to **Tools > Board > Board Manager...**. Here we need to look for the **Arduino SAMD boards (32-bits Arm® Cortex®-M0+)** and install it.

2. Now, we need to install the library needed. If we are using the Cloud Editor, there is no need to install anything. If we are using an offline editor, simply go to **Tools > Manage libraries..**, and search for **WiFiNINA** and install it.

Code Explanation

> *Note: This section is optional, you can find the complete code further down on this tutorial.*

The initialization begins by including the **WiFiNINA** library, afterwards we need to enter our credentials to our network.

```
#include <WiFiNINA.h>

char ssid[] = " ";
char pass[] = " ";
int keyIndex = 0;
int status = WL_IDLE_S
WiFiServer server(80);

WiFiClient client = se

int ledPin = 2;
```

We can then configure the `setup().` Here we set the Serial Communication to 9600, configure the `pinMode` for our LED, and use the line `while(!Serial);` to only initialize the rest of the program as long as we open the Serial Monitor. We do this since important information is printed in the Serial Monitor, and if we upload it, we may risk missing it. Then, we execute two functions: `enable_WiFi()` and `connect_WiFi`, which we will use to connect to our WiFi. We then use `server.begin()` to start hosting the server, once we are connected. The final function, `printWiFiStatus()` simply prints the information about the connection status in the Serial Monitor. Here, we will see the IP address we need to connect to.

```
void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTP
  while (!Serial);

  enable_WiFi();
  connect_WiFi();

  server.begin();
  printWifiStatus();

}
```

The loop of this program is very short. First, we use `client` to check if the `server` is available. If it is, we execute the `printWEB()` function.

```
void loop() {
  client = server.avail

  if (client) {
    printWEB();
  }
}
```

Next up is the functions that we used in `setup()`. These are `printWiFiStatus()`, `enableWiFi()`, and `connect_WiFi()`.

If we look at `printWiFiStatus()`, you can see that it basically prints different things in the Serial Monitor. Most importantly, it prints the board's IP address, which we will need to enter in the browser to control the Arduino.

```
```

```
void printWifiStatus(
  // print the SSID o
  Serial.print("SSID:
  Serial.println(WiFi

  // print your board
  IPAddress ip = WiFi
  Serial.print("IP Ad
  Serial.println(ip);

  // print the receiv
  long rssi = WiFi.RS
  Serial.print("signa
  Serial.print(rssi);
  Serial.println(" dB

  Serial.print("To se
  Serial.println(ip);
}

void enable_WiFi() {
  // check for the Wi
  if (WiFi.status() =
    Serial.println("C
    // don't continue
    while (true);
  }
```

Now, we will look at the core of this program: the `printWEB()` function, which we call from the `loop()`.

Here, we first begin by checking if `client` is available, and if it is, we enter a `while()` loop. Inside the while loop, we will use `client.print` to start printing HTML code that can be viewed from the browser. To not overload the Arduino's board memory, we use a very basic setup: two links that turn an LED either ON or OFF.

> This line of code is used to turn **ON** the LED, by adding **/H** to the end of the URL.

```
client.print("Click <a
href=\"/H\">here</a> turn
the LED on<br>");
```

This line of code is used to turn **OFF** the LED, by adding **/L** to the end of the URL.

```
client.print("Click <a
href=\"/L\">here</a> turn
the LED off<br>");
```

We will also make a reading on an analog pin (A1). Even though we do not have anything connected, we can simply see how we can read something connected to the board, and then print it to the client.

```
client.print("Random reading
from analog pin: ");
```

```
client.print(randomReading);
```

When the printing is done, we use `break;` to exit the while loop. Now, we configure the program to check whether there's a **H** or **L** added to the URL, where we use `digitalWrite(ledPin, STATE)` to turn the LED ON or OFF.

```
void printWEB() {

  if (client) {
    Serial.println("n
    String currentLin
    while (client.con
      if (client.avai
        char c = clie
        Serial.write(
        if (c == '\n'

          // if the c
          // that's t
          if (current

            // HTTP h
            // and a
            client.pr
            client.pr
            client.pr

            //create
            client.pr
            client.pr

            int rando
            client.pr
            client.pr
```

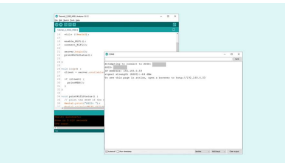
Complete Code

If you choose to skip the code building section, the complete code can be found below:

```
#include <WiFiNINA.h

char ssid[] = "";
char pass[] = "";
int keyIndex = 0;
int status = WL_IDLE
WiFiServer server(80

WiFiClient client =

int ledPin = 2;

void setup() {
  Serial.begin(9600)
  pinMode(ledPin, OU
  while (!Serial);

  enable_WiFi();
  connect_WiFi();

  server.begin();
  printWifiStatus();

}

void loop() {
  client = server.av
```

Upload Sketch and Testing the Program

Once we are finished with the coding, we can upload the sketch to the board. Once it is successful, open the Serial Monitor and it should look like the following image:



Copy the IP address and enter it in a browser. Now, we should see a very empty page with two links at the top left that says **"Click here to turn the LED on"** and **"Click here to turn the LED off"**.

Interacting through a browser.

When interacting with the links, you should see the LED, connected to pin 2, turn on and off depending on what you click. Now we have successfully created a way of interacting with our MKR WiFi 1010 board remotely.

Troubleshoot

If the code is not working, there are some common issues we might need to troubleshoot:

- We have not updated the latest firmware for the board.
- We have not installed the Board Package required for the board.
- We have not installed the **WiFiNINA** library.
- We have entered the SSID and PASS incorrectly: remember, it is case sensitive.
- We have not selected the right port to upload: depending on what computer we use, sometimes the board is duplicated. By simply

Conclusion

In this tutorial, we learned how to create a basic web interface from scratch. We learned how to control an LED remotely, and how to display the value of an analog pin in the browser as well. Using this example, we can build much more complex projects, and if you are familiar with both HTML and CSS, you can create some really cool looking interfaces! If you are new to HTML and CSS, there are plenty of online guides that can guide you, you can visit **w3schools** or **codecademy** for many tips and tricks.

> *Note: The memory of the Arduino MKR WiFi 1010 is not infinite. It is encouraged to use external CSS files if you are planning a bigger project, as it reduces the memory.*

> *Tip: Check out **fontawesome** to get access to thousands of free icons that you can customize your local web server with!*

wrong, you can edit this page here.

Was this article helpful?